CSC111 Assembly Language Programming
Fall 2026 Midterm Exam ANS

Student Name: ID:

Total Points	

Note: This exam paper should be kept confidential and any dissemination violates copyright.

Q1	Q2	Q3	Q4	Q5	Q6
/10	/10	/20	/20	/20	/20

Q1 (10 points) Multiple-choice questions: enter your answer keys here:

1	2	3	4	5	6	7	8	9	10
C	D	A	C	B	A	D	C	B	D

For the following multiple-choice questions, each question has exactly one correct answer key. If multiple choices are correct, choose the option "All of the above". **Fill in the answer keys in the table above. (Answer keys written in the question area will not be counted.)**

1. In a 5-bit system, adding unsigned ints 29 and 5 sets which condition?
   A. No flags set
   B. Overflow flag set
   C. Carry flag set
   D. Zero flag set
   ANS: C (true sum 29 + 5 = 34 exceeds $2^5-1 = 31$, so Carry = 1)
2. In a 5-bit system, subtracting unsigned ints 4 - 7 results in which carry/borrow status?
   A. Carry=1 (Borrow=0)
   B. Carry=1 (Borrow=1)
   C. Carry=0 (Borrow=0)
   D. Carry=0 (Borrow=1)
   ANS: D (4 < 7, so a borrow occurs. ARM defines C = NOT Borrow, therefore C = 0.)
3. Which statement is true when adding the signed ints 12 and 7 in a 5-bit system?
   A. Signed overflow occurs (V = 1)
   B. Carry out occurs (C = 1)
   C. The result is zero (Z = 1)
   D. The result 19 is represented correctly
   ANS: A (signed 12 + 7 = 19 exceeds the 5-bit signed maximum of 15, so V = 1. The binary result 10011 also sets N = 1, but there is no carry out because 19 < 32.)
4. In two's complement, TC(x) can be obtained by:
   A. Invert bits
   B. Invert bits and subtract one
   C. Invert bits and add one
   D. Add one then invert bits
   ANS: C

5. What is the bit width of each register in ARM Cortex-M processors?
   A) 8 bits
   B) 32 bits
   C) 16 bits
   D) 64 bits
   ANS: B

6. Which registers are considered "Low Registers" in ARM Cortex-M and can be accessed by any instruction?
   A) R0-R7
   B) R8-R12
   C) R13-R15
   D) R0-R12
   ANS: A

7. What does ADC r1, r2, r3 compute?
   A. r1 = r2 − r3 + C
   B. r1 = r2 + r3
   C. r1 = r2 + r3 − C
   D. r1 = r2 + r3 + C
   ANS: D (Add with Carry: Rd = Rn + Op2 + C)

8. Which instruction performs reverse subtraction in ARM?
   A. SBC
   B. SUB
   C. RSB
   D. ADC
   ANS: C (RSB (Reverse Subtract) subtracts the first operand (Rn) from the second operand (Operand2): Rd = Operand2 − Rn)

9. What happens when you execute LDRSH r1, [r0] and the halfword at the memory location has value 0x8123?
   A) r1 = 0x00008123
   B) r1 = 0xFFFF8123
   C) r1 = 0x81230000
   D) r1 = 0x00000123
   ANS: B (LDRSH sign-extends; 0x8123 has its top bit set, so it is negative and is extended with 1s)

10. In LDR r1, [r0, r2, LSL #3], what is the effective address used?
    A) r0 + r2
    B) r0 + (r2 × 2)
    C) r0 + (r2 × 4)
    D) r0 + (r2 × 8)
    ANS: D (the index r2 is shifted left by 3, i.e. multiplied by 8)

Q2. (10 points) Binary numbers

Assuming a 4-bit system. Show the equivalent decimal values when the data is interpreted as unsigned binary or signed binary.

Binary Value	Signed Decimal Value	Unsigned Decimal Value
1010		

0101		
1011		
0010		
0100		

ANS:

Binary Value	Signed Decimal Value	Unsigned Decimal Value
1010	-6	10
0101	5	5
1011	-5	11
0010	2	2
0100	4	4

Q3. (20 points) Unsigned and signed arithmetic

Assume a 4-bit system. For each of the following operations, give the equivalent arithmetic calculation in decimal, and the NZCV flags, based on the first row that has been given to you. GT denotes Ground Truth value when there is a carry or overflow.

Operation	Result in binary	Equivalent unsigned arithmetic in decimal	Equivalent signed arithmetic in decimal	NZCV
0101 + 0011	1000	**5 + 3 = 8**	**5 + 3 = -8 (GT=8)**	**1001**
1000 - 0001	0111			
0110 - 1011	1011			
1010 + 1001	0011			
1101 + 0011	0000			
1011 - 0011	1000			

ANS:

Review:

1) N (Negative): 1 if the highest bit of the result is 1.
2) Z (Zero): 1 if the result is 0000.
3) C (Carry): In addition, 1 if unsigned result > 15. In subtraction, 1 if there is NO borrow (i.e. unsigned A $\geqslant$ unsigned B).

4) V (Overflow): 1 if the signed result falls outside the -8 to 7 range.
5) Unsigned 4-bit range: 0 to 15, Signed 4-bit range: −8 to +7.

Operation	Result in binary	Equivalent unsigned arithmetic in decimal	Equivalent signed arithmetic in decimal	NZC V
0101 + 0011	1000	**5 + 3 = 8**	**5 + 3 = -8 (GT=8)**	**1001**
1000 - 0001	0111	8 - 1 = 7	-8 - 1 = 7 (GT=-9)	0011
0110 - 1011	1011	6 - 11 = 11 (GT=-5)	6 - -5 = -5 (GT=11)	1001
1010 + 1001	0011	10 + 9 = 3 (GT=19)	-6 + -7 = 3 (GT=-13)	0011
1101 + 0011	0000	13 + 3 = 0 (GT=16)	-3 + 3 = 0	0110
1011 - 0011	1000	11 - 3 = 8	-5 - 3 = -8	1010

Note: for the row 1101 + 0011 = 0000, the unsigned result wraps around to 0 with a carry out (C=1, GT=16), so Z=1 and C=1. The signed result −3 + 3 = 0 is correct, so V=0.

Grading rubric (20 points): 4 points per row × 5 rows = 20 points. In each of the 5 rows to be filled in, each arithmetic box (unsigned and signed) is worth 1.5 point and the NZCV flag box is worth 1 point. Total point is rounded up to integer.

Detailed explanation of each row (4-bit system):

How the flags are computed: N = 1 if bit 3 of the 4-bit result is 1. Z = 1 if the 4-bit result is 0000. C = 1 if an addition produces a carry out of bit 3 (unsigned sum > 15); for a subtraction A − B, C = 1 if there is NO borrow, i.e., unsigned A ≥ unsigned B (the hardware computes A + NOT(B) + 1, and C is the carry out of that addition). V = 1 if the signed result falls outside −8 to +7 (equivalently, adding two numbers of the same sign gives a result of the opposite sign). The unsigned interpretation uses the range 0 to 15; the signed (two's complement) interpretation uses the range −8 to +7, where 1000 = −8, 1010 = −6, 1001 = −7, 1011 = −5, 1101 = −3. GT denotes the true (mathematical) value before it wraps around to 4 bits.

Row 1 (given): 0101 + 0011 = 1000. Unsigned: 5 + 3 = 8, which fits in 0 to 15, so there is no carry (C = 0). Signed: 5 + 3 = -8 (GT = 8). The true sum 8 is above +7, so the 4-bit result wraps to -8 and the overflow flag is set (V = 1). The result 1000 has bit 3 = 1, so N = 1, and it is not zero, so Z = 0. NZCV = 1001.

Row 2: 1000 - 0001 = 0111. Unsigned: 8 - 1 = 7. Since 8 ≥ 1 there is no borrow, so C = 1 (check: 1000 + 1110 + 1 = 1 0111, carry out = 1). Signed: -8 - 1 = 7 (GT = -9). 1000 is -8 and the true result -9 is below -8, so the 4-bit result wraps to +7 and V = 1. The result 0111 has bit 3 = 0, so N = 0, and it is not zero, so Z = 0. NZCV = 0011.

Row 3: 0110 - 1011 = 1011. Unsigned: 6 - 11 = 11 (GT = -5). The true result -5 is below 0, so it wraps to 11, and since 6 < 11 a borrow occurs, so C = 0 (check: 0110 + 0100 + 1 = 1011, no carry out). Signed: 6 - -5 = -5 (GT = 11). 1011 is -5 and the true result 11 is above +7, so the 4-bit result wraps to -5 and V = 1. The result 1011 has bit 3 = 1, so N = 1, and it is not zero, so Z = 0. NZCV = 1001.

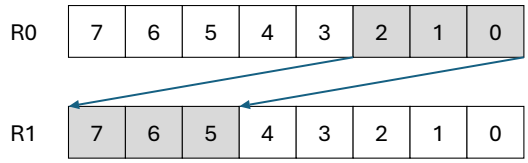
Row 4: 1010 + 1001 = 0011. Unsigned: 10 + 9 = 3 (GT = 19). The true sum 19 exceeds 15, so it wraps to 19 - 16 = 3 with a carry out, C = 1. Signed: -6 + -7 = 3 (GT = -13). 1010 is -6 and 1001 is -7; the true result -13 is below -8, so the 4-bit result wraps to +3 (two negative operands gave a positive result) and V = 1. The result 0011 has bit 3 = 0, so N = 0, and it is not zero, so Z = 0. NZCV = 0011.

Row 5: 1101 + 0011 = 0000. Unsigned: 13 + 3 = 0 (GT = 16). The true sum 16 wraps to 0 with a carry out, so C = 1 and Z = 1. Signed: -3 + 3 = 0. 1101 is -3, and the result 0 is correct and in range, so V = 0 (a positive and a negative operand can never overflow). The result 0000 has bit 3 = 0, so N = 0. NZCV = 0110.

Row 6: 1011 - 0011 = 1000. Unsigned: 11 - 3 = 8. Since 11 ≥ 3 there is no borrow, so C = 1 (check: 1011 + 1100 + 1 = 1 1000, carry out = 1). Signed: -5 - 3 = -8. 1011 is -5, and -8 is exactly the minimum of the signed range, so the result is correct and V = 0. The result 1000 has bit 3 = 1, so N = 1, and it is not zero, so Z = 0. NZCV = 1010.

Q4. (20 points) Bit manipulations

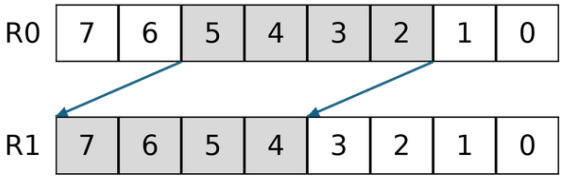
(a) (5 points) Assume an 8-bit system (only the low 8 bits of R0 and R1 are meaningful). Copy the lowest three bits 2..0 in R0 to bits 7..5 in R1, and keep all other bits in R1 unchanged. (Arithmetic expressions are optional, ARM assembly code is mandatory.)



ANS: Arithmetic:
R1 = (R1 & 0b00011111) | ((R0 & 0b00000111) << 5)
Or R1 = (R1 & 0x1F) | ((R0 << 5) & 0xE0)

ARM Assembly:
AND R1, R1, #0x1F ; clear destination bits 7..5
AND R2, R0, #0x07 ; mask source bits 2..0
ORR R1, R1, R2, LSL #5 ; shift left and OR in one instruction
Alternative (shift first, then clear the destination field with BIC):
AND R2, R0, #0x07 ; mask source bits 2..0
LSL R2, R2, #5 ; move them to bits 7..5
BIC R1, R1, #0xE0 ; clear destination bits 7..5 (0xE0 = 0b11100000)
ORR R1, R1, R2 ; insert the three bits

(b) (5 points) Assume an 8-bit system (only the low 8 bits of R0 and R1 are meaningful). Copy bits 5..2 in R0 to bits 7..4 in R1, and keep R1[3:0] unchanged.



ANS: Arithmetic:
R1 = (R1 & 0b00001111) | ((R0 & 0b00111100) << 2)
Or R1 = (R1 & 0x0F) | ((R0 & 0x3C) << 2)

ARM Assembly:
AND R1, R1, #0x0F ; R1 = R1 & 0b00001111 (clear bits 7..4)

```
AND R2, R0, #0x3C       ; mask source bits 5..2 of R0 (0b00111100)
ORR R1, R1, R2, LSL #2  ; shift left 2 and OR into R1
Alternative (shift first, then clear the destination field with BIC):
AND R2, R0, #0x3C       ; mask source bits 5..2 of R0 (0b00111100)
LSL R2, R2, #2          ; move them to bits 7..4
BIC R1, R1, #0xF0       ; clear destination bits 7..4 (0xF0 = 0b11110000)
ORR R1, R1, R2          ; insert the four bits
```

(c) (10 points) Assume a 32-bit system. R0 and R1 are initialized with the values below. Write a sequence of assembly instructions that would produce the values in registers R2–R4 from R0 and R1. Do not use MOVW + MOVT. Do not use the pseudo-instruction LDR to load large constants into a register.

Given:

R0	0x1234ABCD
R1	0x0000FFFF

Produce:

R2	0x12340000
R3	0xABCD1234
R4	0xFFFF1234

ANS:

```
BIC R2, R0, R1
ROR R3, R0, #16
MVN R4, R1
ORR R4, R4, R0, LSR #16
```

Alternative sequence for R4:

```
LSR R4, R0, #16
ORN R4, R4, R1
```

Another alternative sequence for R4:

```
LSL R4, R1, #16
ORR R4, R4, R0, LSR #16
```

Alternative sequence for R3 (the primary solution above uses ROR):

```
LSL R3, R0, #16
ORR R3, R3, R0, LSR #16
```

Alternative sequence for R2:

```
LSR R2, R0, #16
LSL R2, R2, #16
```

Explanation (optional):

1. Instruction: **BIC R2, R0, R1**
   - Bit Clear: R2 = R0 AND (NOT R1), i.e., it clears the bits of R0 that are 1 in R1.
   - R2 = 0x1234ABCD & ~0x0000FFFF = 0x1234ABCD & 0xFFFF0000 = 0x12340000
   - (Note: only the upper halfword 0x1234 survives.)
2. Instruction: **ROR R3, R0, #16**
   - Rotate Right by 16 bits: bits shifted out of the low end re-enter at the top, so the two halfwords swap.
   - 0x1234ABCD rotated right by 16 bits = 0xABCD1234
   - R3 = 0xABCD1234
3. Instruction: **MVN R4, R1** followed by **ORR R4, R4, R0, LSR #16**
   - **Step 1:** MVN inverts all 32 bits of R1:
     R4 = ~0x0000FFFF = 0xFFFF0000
   - **Step 2:** the barrel shifter computes R0 LSR #16 = 0x00001234, then ORR merges it with R4:
     R4 = 0xFFFF0000 | 0x00001234 = 0xFFFF1234
   - Hence, **R4 = 0xFFFF1234**.

Explanation of the alternative sequences:

- LSR R4, R0, #16 ; ORN R4, R4, R1 → R4 = 0x00001234, then R4 OR (NOT R1) = 0x00001234 | 0xFFFF0000 = 0xFFFF1234 (Note: ORN is a Thumb-2 instruction available on Cortex-M3/M4; it does not exist in classic A32 ARM.)
- LSR R2, R0, #16 ; LSL R2, R2, #16 → shifting right then left by 16 clears the low halfword: R2 = 0x12340000
- LSL R4, R1, #16 ; ORR R4, R4, R0, LSR #16 → R1 LSL #16 = 0xFFFF0000 (the low halfword 0xFFFF moves to the top), then R0 LSR #16 = 0x00001234 is ORed in: R4 = 0xFFFF0000 | 0x00001234 = 0xFFFF1234.
- LSL R3, R0, #16 ; ORR R3, R3, R0, LSR #16 → R0 LSL #16 = 0xABCD0000 (the low halfword moves to the top), then R0 LSR #16 = 0x00001234 (the high halfword moves to the bottom) is ORed in: R3 = 0xABCD0000 | 0x00001234 = 0xABCD1234. This does the same job as ROR #16 using two shifts and an OR.
- Common incorrect answer for R2: AND R2, R0, #0xFFFF0000. The result would be the right value (0x1234ABCD & 0xFFFF0000 = 0x12340000), but the instruction is not valid and the assembler rejects it, because 0xFFFF0000 cannot be encoded as an immediate operand. An ARM data-processing immediate must be an 8-bit value rotated to some position, or one of the byte patterns 0x00XY00XY, 0xXY00XY00 or 0xXYXYXYXY; 0xFFFF0000 has 16 consecutive 1 bits, which is more than 8 significant bits, so it is not one of these forms. (The BIC form BIC R2, R0, #0x0000FFFF fails for the same reason.) A large constant must not be built with MOVW + MOVT or LDR Rd, =const in this question, so R2 has to be formed from registers that already hold the needed value: use R1 = 0x0000FFFF as the mask (BIC R2, R0, R1), or clear the low halfword with the shift pair LSR #16 then LSL #16.

Q5. (20 points) Program execution tracing

Assume a 32-bit system with **little-endian** byte order. The following assembly program operates on memory and registers. Fill in the values of registers R1 through R7 and the final memory contents after the entire program has finished executing. Write your answers in hexadecimal format.

Assembly program:

```
LDR   R1, =0x20000000
LDR   R2, [R1, #4]
LDRH  R3, [R1], #4
ASR   R4, R2, #8
MOV   R5, #0x0F
ORR   R6, R3, R5, LSL #8
EOR   R7, R2, R5
STR   R7, [R1, #4]
```

Initial memory contents:

Addr	Content															
0x20000000	11	22	33	44	9A	BC	DE	F0	55	66	77	88	99	AA	BB	CC

Final register values:

R1	R2	R3	R4
R5	R6	R7	

Final memory contents:

Addr	Content															
0x20000000																

ANS:

Final register values:

R1	R2	R3	R4
0x20000004	0xF0DEBC9A	0x00002211	0xFFF0DEBC
R5	R6	R7	
0x0000000F	0x00002F11	0xF0DEBC95	

Final memory contents:

Addr	Content															
0x20000000	11	22	33	44	9A	BC	DE	F0	95	BC	DE	F0	99	AA	BB	CC

Grading rubric (20 points): 2 points for each of R1–R7 (14 points in total) and 6 points for the final memory contents.

Explanations (not required in exam):

1. **LDR R1, =0x20000000** → Initializes R1 with the base memory address 0x20000000.
2. **LDR R2, [R1, #4]** → Offset addressing (R1 is not changed). Loads the 4 bytes at 0x20000004 (9A BC DE F0). In little-endian format the lowest address holds the least significant byte, so R2 = 0xF0DEBC9A.
3. **LDRH R3, [R1], #4** → Post-indexed load of a halfword. It first loads the 2 bytes at 0x20000000 (11 22), which in little-endian format form 0x2211, zero-extended to R3 = 0x00002211. Then R1 is updated: R1 = 0x20000000 + 4 = 0x20000004.
4. **ASR R4, R2, #8** → Arithmetic shift right of 0xF0DEBC9A by 8 bits. The sign bit (1) is replicated into the top 8 bits, so R4 = 0xFFF0DEBC (a logical shift would give 0x00F0DEBC).
5. **MOV R5, #0x0F** → R5 = 0x0000000F.
6. **ORR R6, R3, R5, LSL #8** → The barrel shifter first shifts R5 left by 8 bits (0x00000F00), then the result is ORed with R3 (0x00002211): R6 = 0x00002F11.
7. **EOR R7, R2, R5** → Bitwise exclusive OR: 0xF0DEBC9A ^ 0x0000000F = 0xF0DEBC95 (since 0x9A ^ 0x0F = 0x95). R7 = 0xF0DEBC95.
8. **STR R7, [R1, #4]** → Stores R7 at address R1 + 4 = 0x20000004 + 4 = 0x20000008 (R1 is unchanged). In little-endian format it writes the bytes 95 BC DE F0 into 0x20000008 through 0x2000000B, replacing 55 66 77 88.

Alternative answer if big-endian byte order is assumed:

The question states little-endian byte order. If a student instead assumes big-endian byte order (the byte at the lowest address is the most significant byte) and gets the following answer completely correct, deduct 4 points for not following the stated byte order. Big-endian answer:

Final register values (big-endian):

R1	R2	R3	R4
0x20000004	0x9ABCDEF0	0x00001122	0xFF9ABCDE
R5	R6	R7	
0x0000000F	0x00001F22	0x9ABCDEFF	

Final memory contents (big-endian):

Addr	Content															
0x20000000	11	22	33	44	9A	BC	DE	F0	9A	BC	DE	FF	99	AA	BB	CC

Big-endian trace: LDR R2, [R1, #4] loads 9A BC DE F0, so R2 = 0x9ABCDEF0. LDRH R3, [R1], #4 loads the halfword at 0x20000000 (11 22), so R3 = 0x00001122, and then R1 = 0x20000004. ASR R4, R2, #8 gives 0xFF9ABCDE. R6 = 0x00001122 | 0x00000F00 = 0x00001F22. R7 = 0x9ABCDEF0 ^ 0x0000000F = 0x9ABCDEFF. STR R7, [R1, #4] stores 9A BC DE FF at 0x20000008 (big-endian order), so the last 8 bytes of memory become 9A BC DE FF 99 AA BB CC.

Q6. (20 points) Assume a 32-bit system. Consider an array[] of 16 unsigned integers (each integer is 4 bytes). Assume register R0 holds the base memory address of array, i.e., array[0]. A compiler associates variables z and y with registers R1 and R2, respectively. Translate this C program into ARM assembly language based on the provided comments. Use R3 as a temporary register. (Note: Assume z is an **unsigned integer**. You may choose to use either pre-index, or pre-index with update, or post-index addressing.)

C program:

```
uint32_t z = y - array[1];
array[2] = z * 4;
array[3] = z / 8;
array[4] = z * 9;
array[5] = z & 0xFF;
```

Assembly program:

_____; load array[1] into temporary register R3

_____; z = y - array[1]

_____; R3 = z * 4

_____; store into array[2]

_____; R3 = z / 8

_____; store into array[3]

_____; R3 = z * 9

_____; store into array[4]

_____; R3 = z & 0xFF

_____; store into array[5]

ANS: Grading rubric (20 points): the assembly program has 10 entries (blanks), and each entry is worth 2 points.

Version 1: Using pre-index addressing

```
LDR  R3, [R0, #4]       ; load array[1] (offset 1*4 = 4) into a temp register
SUB  R1, R2, R3         ; z = y - array[1]
LSL  R3, R1, #2         ; R3 = z * 4
STR  R3, [R0, #8]       ; store into array[2] (offset 2*4 = 8)
LSR  R3, R1, #3         ; R3 = z / 8
STR  R3, [R0, #12]      ; store into array[3] (offset 3*4 = 12)
ADD  R3, R1, R1, LSL #3  ; R3 = z + 8z = z * 9
STR  R3, [R0, #16]      ; store into array[4] (offset 4*4 = 16)
AND  R3, R1, #0xFF      ; R3 = z & 0xFF
STR  R3, [R0, #20]      ; store into array[5] (offset 5*4 = 20)
```

Version 2: Using pre-index with update addressing.

```
LDR  R3, [R0, #4]!      ; load array[1] and update R0 to point to array[1]
SUB  R1, R2, R3         ; z = y - array[1]
LSL  R3, R1, #2         ; R3 = z * 4
STR  R3, [R0, #4]!      ; update R0 to array[2], store z * 4
LSR  R3, R1, #3         ; R3 = z / 8
STR  R3, [R0, #4]!      ; update R0 to array[3], store z / 8
ADD  R3, R1, R1, LSL #3  ; R3 = z * 9
STR  R3, [R0, #4]!      ; update R0 to array[4], store z * 9
AND  R3, R1, #0xFF      ; R3 = z & 0xFF
STR  R3, [R0, #4]!      ; update R0 to array[5], store z & 0xFF
```

Version 3: Using post-index addressing.

```
LDR  R3, [R0, #4]       ; load array[1] into R3
SUB  R1, R2, R3         ; z = y - array[1]
ADD  R0, R0, #8         ; increment R0 to point to array[2]
LSL  R3, R1, #2         ; R3 = z * 4
STR  R3, [R0], #4       ; store into array[2], then update R0 to array[3]
LSR  R3, R1, #3         ; R3 = z / 8
STR  R3, [R0], #4       ; store into array[3], then update R0 to array[4]
ADD  R3, R1, R1, LSL #3  ; R3 = z * 9
STR  R3, [R0], #4       ; store into array[4], then update R0 to array[5]
AND  R3, R1, #0xFF      ; R3 = z & 0xFF
STR  R3, [R0]           ; store into array[5]
```

Why MUL R3, R1, #9 (or UDIV/SDIV R3, R1, #8) is not a valid answer:

In ARM Cortex-M (Thumb-2), MUL, UDIV and SDIV only have register forms: MUL Rd, Rn, Rm and UDIV Rd, Rn, Rm. They do not accept an immediate operand: MUL R3, R1, #9 or UDIV R3, R1, #8 are invalid syntax, so the constant must first be assigned to a register, e.g., MOV R4, #9 then MUL R3, R1, R4. Multiplying by 9 can also be done with one instruction: ADD R3, R1, R1, LSL #3. For the division by 8, because z is an unsigned integer, LSR R3, R1, #3 is the mathematically correct shift.